

HORIZON

A predictive network world model for
attack trajectory forecasting

Problem Statement 26153 — AI based Network Attack Forecasting from Network Traffic Data

Organisation National Technical Research Organisation (NTRO)

Theme Blockchain & Cybersecurity · **Category** Software

Status Draft for build · **Date** September 2026

We are not building an intrusion detector. We are building a learned simulator of network behaviour that generates network states it has never observed, and reports how likely those imagined futures are to reach compromise — with a horizon, a confidence band, and a lead time measured in seconds.

1. Problem & Objective

A conventional IDS computes $f(x_t) \rightarrow \{\text{benign}, \text{malicious}\}$. It fires when the evidence is already present, which by definition means the behaviour has already occurred. An infiltration, however, is a process unfolding over time: ports are probed in a sequence, SYN flags precede ACK floods, reconnaissance timing shifts before lateral movement begins. Treating each flow in isolation discards that structure entirely.

HORIZON models the process instead of the packet. It compresses traffic telemetry into windowed per-host **states**, learns the stochastic **transition dynamics** that govern how those states evolve, and rolls them forward K steps to forecast whether the current trajectory is converging toward compromise — before the attacker completes the kill chain.

1.1 Goals

Goal	Success condition
Learn transition dynamics	A trained model of $P(S_{t+1} \mid h_t)$ whose predicted states can be scored against ground truth via per-feature nRMSE at every horizon.
Forward simulation	Recursive K-step rollout consuming no real data after time t ; risk and stage computed only from predicted states.
Prove dynamics are real	Persistence and time-shuffle ablations cause measurable collapse in lead time and AUC.
Early warning	Median forecast lead time > 60 s before labelled attack-stage onset, at a deployable false-positive rate.
Generalisation	Risk rises before onset for an attack class held out entirely from training.
Interpretability	Every forecast ships with signed feature attributions, a driving time window, and a counterfactual.
Offline operation	Full stack runs air-gapped via <code>docker compose up</code> . Zero cloud API calls.

1.2 Non-goals

- **Not a causal simulator.** The data is observational. We learn temporal dynamics, not causal structure. Counterfactual rollouts are model-internal what-ifs, not interventional claims.
- **Not an alert-correlation or investigation platform.** The output is a forecast with a time axis, not a reconstructed incident graph.
- **Not real-time packet capture.** Offline replay of PCAP/CSV at accelerated wall-clock. Live tap is a deployment slide, not v1 code.
- **Not ATT&CK technique-level prediction.** We map dataset labels to tactics for presentation. The dataset does not support technique-level ground truth.

1.3 The distinction that defines the product

	Traditional IDS	HORIZON
Input	Current flow or packet	Sequence of 30 windowed host states
Core object	Decision boundary	Transition operator
Learns	$P(\text{label} \mid x)$	$P(S_{t+1} \mid h_t)$ + a risk readout
Produces	A label	A simulated trajectory with a time axis
Fires	When evidence exists	Before evidence exists

Uncertainty	Class confidence	Distribution over futures, widening with horizon
Answers "what if"	No	Yes — counterfactual rollout
Fails by	Missing novel signatures	Accumulating rollout error
Evaluated by	F1, FPR	F1, FPR <i>and</i> lead time, calibration, state error

2. System Architecture

Eight components across two planes. The **offline plane** (§8 in the diagram) runs once and produces model weights plus a frozen scaler. The **serving plane** is an event-driven pipeline of four stateless Python services joined by Redis Streams, fronted by FastAPI and a Next.js dashboard. Every stage is causally ordered: no component may read data whose timestamp exceeds the current window.

DESIGN RULE

The inference service is stateless. Sequence context arrives on the stream, forecasts leave on the stream. Nothing is held in process memory between messages. This is what makes the horizontal scaling claim in §9 true rather than aspirational.

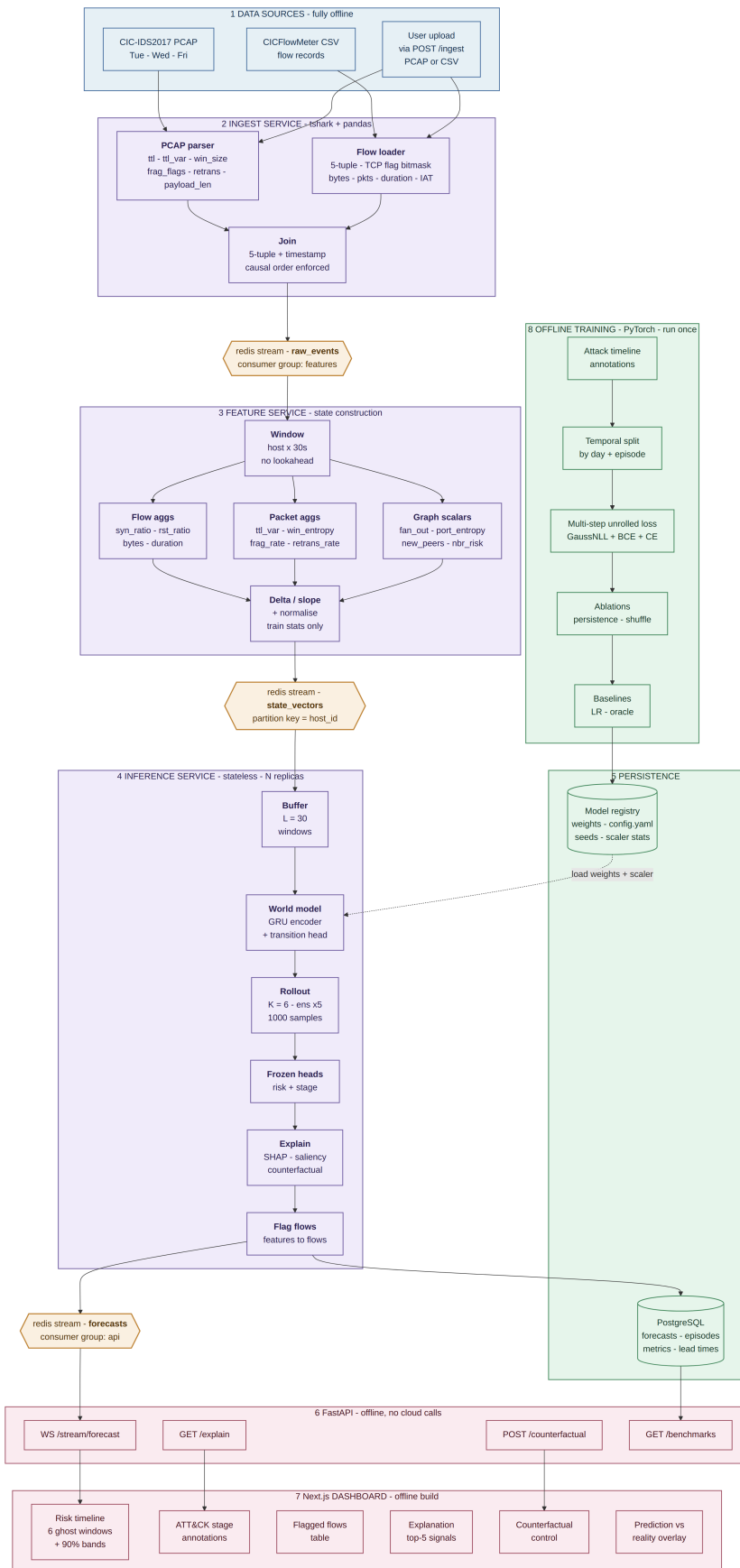


Figure 1 — HORIZON system architecture. Data sources feed a two-level ingest (PCAP via tshark for packet features, CSV for flow features) joined on five-tuple and timestamp. The feature service constructs per-host 30-second state vectors. The stateless inference service runs the world model and K-step rollout. Redis Streams partitioned by host_id decouple every stage and provide the natural scale-out boundary. The offline training plane on the right produces the weights and scaler statistics consumed at serving time.

3. The World Model

The world model is the core deliverable and the thing the problem statement is most emphatic about: *a learned model of state transition dynamics, not a static classifier*. Three design choices make that claim falsifiable rather than rhetorical.

3.1 The frozen-head design

The risk head and stage head are trained **only on observed states**, then frozen. At inference they are applied to states the transition model invented. Consequently, **every unit of forward-looking capability the system has must originate in the transition model**. There is no other path. This is not a convenience — it is the architectural commitment that makes the persistence ablation a genuine falsification test rather than a formality.

3.2 Recursive latent rollout

We roll forward recursively, feeding each prediction back as if observed. Direct multi-horizon regression would score better at long horizons but produces no reusable transition operator — and without an operator there is no counterfactual, no simulation, and no defensible world-model claim. Training uses a **multi-step unrolled loss**: unroll K steps during training and sum loss over all of them, so the model learns robustness to its own error. Scheduled sampling is available if training destabilises.

3.3 Distributional output

$P(S_{t+1} \mid S_t)$ is a distribution, not a point. The transition head emits mean and log-variance per feature. Combined with a 5-seed deep ensemble, we sample ~ 1000 trajectories per forecast and report the fraction crossing a risk threshold — which is a far more honest output than a point estimate, and visually far stronger in the demo.

HONESTY CONSTRAINT

Raw observations are **not** Markov — a single window cannot tell you whether SYN ratio is rising or falling. The usable form is $P(S_{t+1} \mid h_t)$ where $h_t = f(S_{1..t})$. The recurrent hidden state is what carries the Markov property. We state this explicitly rather than pretending single-step Markovianity holds.

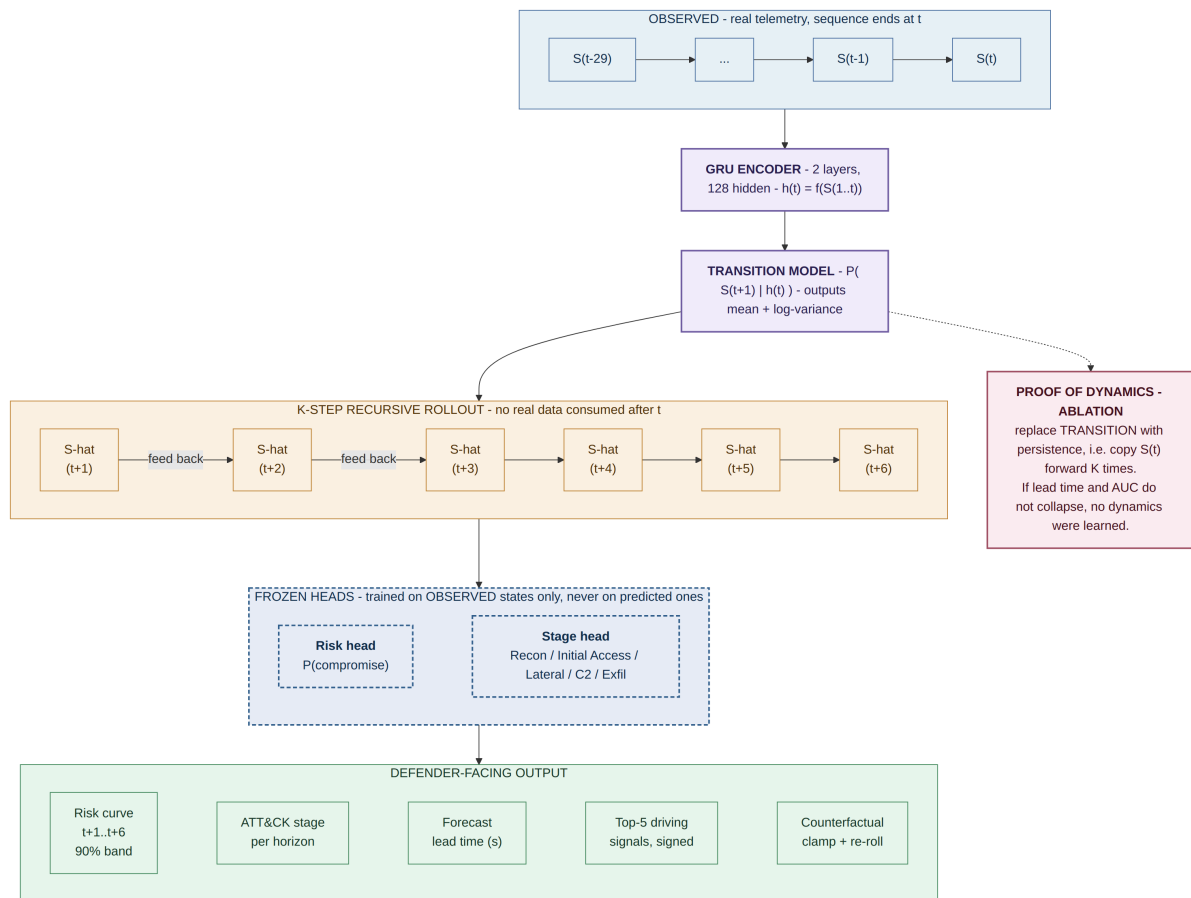


Figure 2 — World model internals and K-step rollout. Observed states through time t are encoded to h_t . The transition model emits a distribution over S_{t+1} . Rollout proceeds recursively for $K=6$ windows with no real data consumed after t . Frozen heads — trained exclusively on observed states — are applied to the predicted states, which is why the persistence ablation on the right constitutes a real test of whether dynamics were learned.

4. Data & State Specification

4.1 Why both feature levels are mandatory

Flow-level features answer *who is talking to whom, how much, how often* — this is where campaign structure lives (fan-out, port spread, directional asymmetry). Packet-level features answer *how it is talking* — this is where mechanism leaks through. A flow record for a legitimate backup and a slow exfiltration can look identical; the packet-level shape distinguishes them. Conversely, packet features alone cannot reveal that a host has begun touching forty new peers.

CRITICAL BUILD DEPENDENCY

TTL variance, TCP window size, IP fragment flags, payload size distribution and retransmission counts **do not exist in CICFlowMeter CSVs**. They must be extracted from raw PCAP. This is the single highest-risk item in the plan and work starts here, not on the model. Use `tshark -T fields` rather than PyShark or Scapy — both are too slow for a full capture day.

4.2 Dataset selection

CIC-IDS2017, Tuesday / Wednesday / Friday. The problem statement names CIC-IDS2018, but its raw captures run to hundreds of gigabytes and will consume the build window. CIC-IDS2017 PCAPs are a few tens of GB per day, are explicitly listed in the permitted dataset section, and

provide a clean day-boundary temporal split plus a labelled Infiltration episode for the held-out generalisation test. The pipeline additionally accepts 2018 CSVs for the flow half.

4.3 State definition

Scope: per host. **Window Δ :** 30 s. **Context L:** 30 windows (15 min). **Horizon K:** 6 windows (3 min). Roughly 45 named features per cell.

Group	Source	Representative features
Flow aggregates	CSV / NetFlow	syn_ratio, ack_ratio, rst_ratio, fin_ratio, psh_ratio, urg_ratio, bytes_total, bytes_up_down_ratio, pkts_per_flow, flow_duration_mean/var, iat_mean, iat_var, iat_max, bidirectional_ratio, active_flow_count
Packet aggregates	PCAP / tshark	ttl_mean, ttl_var, tcp_window_mean, tcp_window_entropy, frag_flag_rate, payload_size_mean/var/p95, payload_size_entropy, retrans_count, retrans_rate, seq_port_access_score
Graph scalars	Derived	out_degree, in_degree, dst_ip_entropy, dst_port_entropy, new_peer_count, neighbour_risk_fraction, local_clustering_coeff, reciprocity
Dynamics	Derived	Backward-looking deltas and 3-window rolling slopes for the ~12 most discriminative features above

4.4 Why a feature vector, not a graph

The task is **forward simulation**. Simulating a fixed-dimensional vector is well-posed; simulating an evolving graph requires predicting future structure — node and edge births and deaths over a set that changes every window — which is a research problem, not a hackathon problem. The vector state also gives an exactly matched comparison against the mandated logistic-regression baseline, supports SHAP over named features, and makes per-host lead-time alignment clean.

The relational signal is not discarded. It is preserved through the **graph-derived scalars** in §4.3, computed per host per window from the communication graph. The graph is a feature extractor, never the state.

4.5 Leakage controls

- Window w uses only events with timestamps inside w . Smoothing is backward-looking only.
- Normalisation statistics are fitted on the **training period only** and serialised into the model registry alongside the weights. This is the single most common source of silent leakage.
- Input sequences end at t inclusive; labels describe only $(t, t+K]$.
- Episode-level splitting — no windows from the same attack episode appear on both sides of a split.

4.6 Worked example — a scan escalating

Host `192.168.10.50`, $\Delta = 30$ s, six observed windows:

Window	syn_ratio	dst_port_entropy	unique_dst	iat_var	retrans_rate	new_peer_count
t-5	0.11	1.2	3	0.31	0.01	0
t-4	0.14	1.6	5	0.29	0.01	1
t-3	0.23	2.9	14	0.12	0.04	9
t-2	0.41	4.1	38	0.05	0.09	24
t-1	0.58	4.8	61	0.03	0.14	39
t	0.66	5.0	74	0.02	0.17	47

What the model learns from thousands of such sequences is a **regime signature**, not a threshold. Rising SYN ratio and port entropy combined with *collapsing* inter-arrival variance is machine-driven enumeration — humans and applications produce jittery timing, tools do not. That IAT-variance collapse is one of the more discriminative signals and is precisely what a per-flow classifier misses.

Rolling forward, the transition model has seen that this regime typically resolves one of two ways: the scan saturates and terminates (benign vulnerability scanner), or port entropy narrows sharply while upstream bytes to one or two destinations rise — the transition into exploitation. The forecast is a distribution over those branches. **The model is not detecting the scan. It is predicting which branch the scan takes.**

5. Interfaces

5.1 State vector contract

Defined once as a Pydantic model and imported by every service; the TypeScript type is generated from the OpenAPI spec. Feature-name drift between the training pipeline and the SHAP display is the bug that eats an afternoon late in the build, and it is entirely avoidable.

```
class StateVector(BaseModel):
    host_id: str
    window_ts: datetime # window start, UTC
    features: dict[str, float] # ~45 named scalars, order fixed by FEATURE_ORDER
    schema_ver: str

class Forecast(BaseModel):
    host_id: str
    origin_ts: datetime # time t — nothing after this was observed
    horizons: list[HorizonPoint] # k = 1..6
    lead_time_s: float | None # None if threshold never crossed
    observed_stage: str
    top_signals: list[SignalAttribution] # name, shap_value, direction
    driving_window: int # index of most influential past window

class HorizonPoint(BaseModel):
    k: int; ts: datetime
    p_compromise: float
    ci_low: float; ci_high: float # 90% band from ensemble + sampling
    stage_dist: dict[str, float] # ATT&CK tactic → probability
    predicted_features: dict[str, float] # decoded S-hat, for overlay + explanation
```

5.2 API surface

Endpoint	Purpose
POST /ingest	Upload PCAP or CSV; kicks off parse and publishes to <code>raw_events</code> . Returns a job id.
WS /stream/forecast	Live replay channel. Streams <code>Forecast</code> objects as windows are processed.
GET /explain/{host}/{ts}	Full attribution bundle: SHAP over current state, SHAP over predicted state, temporal saliency, flagged flows.
POST /counterfactual	Clamp one named feature to a value, re-run rollout, return the modified risk curve alongside the original.
GET /benchmarks	Baseline comparison table, ablation results, horizon curves, calibration data.

5.3 Dashboard requirements

The problem statement requires the interface to display three specific things: the infiltration probability timeline, **flagged flows**, and attack stage annotations. Flagged flows do not fall out of a per-host state model natively and require an explicit bridge.

Panel	Content
Risk timeline	Observed risk to t , then six ghost windows with 90% bands widening by horizon. Threshold crossing marked; lead time annotated in seconds.
Stage annotations	Predicted ATT&CK tactic per horizon with confidence. Labelled as a curated mapping from dataset labels, not technique-level ground truth.
Flagged flows	Bridge: when a forecast crosses threshold, resolve the top-ranked SHAP features back to the specific flows in the driving windows that contributed to them. A lookup, not a second model — but it must be built.
Explanation	Top-5 signed signals with magnitude and direction, plus the driving time window from temporal saliency.
Counterfactual	Clamp control per feature. Re-rolls and overlays the modified curve. Labelled "model-internal what-if".
Reality overlay	Once real data for $t+k$ arrives, overlay observed state on the earlier prediction. This is the demo's proof moment.

6. Explainability

Three distinct questions require three distinct mechanisms. Conflating them is the most common weakness in submissions of this type.

Question	Mechanism	Output form
Why is the current state assessed this way?	KernelSHAP over observed S_t against the risk head	"syn_ratio +0.31, new_peer_count +38 drive current risk 0.61"
Why is this future predicted?	Input-gradient saliency $\partial S_{\hat{t}(t+K)} / \partial S(t-j)$ over past windows (Captum)	"Forecast driven primarily by the change between windows $t-3$ and $t-1$ "
Why this stage ?	SHAP on the stage head applied to the <i>predicted</i> state	"Initial Access predicted because projected port entropy narrows while projected upstream bytes to one destination rise"
What if this behaviour stopped?	Counterfactual rollout — clamp a feature, re-simulate	"If new_peer_count stops growing, projected risk at $t+5$ falls $0.85 \rightarrow 0.31$ "

The third row is only expressible because the transition model decodes to named features rather than an opaque latent. The fourth row is **the capability no classifier can offer at all** — it directly exercises the world model as a simulator, and it is the most persuasive element of the demo.

7. Evaluation

7.1 Splits — why random splitting is disqualifying

Adjacent windows overlap heavily in content; a random split places window t in train and $t+1$ in test, which is near-duplication. Worse, a single attack episode gets split across both sides, so the

model memorises that specific episode. Reported F1 approaches 0.99 and means nothing. Judges who know this will ask, and it is the fastest available route to losing credibility.

Split	Design	What it demonstrates
Temporal primary	Train on earlier days, test on later. CIC-IDS2017's day structure makes this natural.	No forward information leakage
Episode-level	No windows from one attack episode on both sides	No episode memorisation
Attack-type holdout key claim	Remove Infiltration entirely from training; evaluate on it	Learned escalation dynamics, not signatures — this answers the "generalise to unseen attacks" requirement directly
Cross-dataset stretch	Train CIC-IDS2017, evaluate UNSW-NB15 on a harmonised feature subset	Transfer. Expect degradation; report it. Honest degradation beats suspicious perfection.

7.2 Baselines — four, not one

Baseline	Role
LR on S_t mandated	Required by the problem statement. Headline comparison row. Isolates the value of temporal modelling on identical features.
LR on flattened $S(t-L..t)$	The sharp one. Has history but no dynamics. Beating this is the real test.
Persistence	Assume $S_{t+k} = S_t$. Baseline for state forecast error and the core ablation.
Oracle detector	Sees S_{t+k} directly. Upper bound. Our forecaster should sit between the flattened-LR and the oracle.

7.3 Metrics

Standard

F1, precision, recall, FPR at a fixed operating point, and AUC-PR (classes are heavily imbalanced). FPR is additionally reported as **alerts per hour per host** — the number that actually decides whether a SOC would deploy this.

Forecast-specific — where the differentiation lives

Metric	Definition
Forecast lead time	For each true stage onset at T , the earliest $t < T$ where predicted risk crossed threshold and stayed above for m consecutive windows . Lead time = $T - t$, in seconds. Report median and full distribution, never the maximum. The persistence requirement is what prevents a single noisy spike counting as a prediction.
Early-warning precision @ k	Of alarms raised at horizon k , the fraction followed by a real attack within k windows. Reported as a curve over k — this shows the true cost of looking further ahead.
Detection-before-stage-completion	Fraction of attack episodes where an alarm fired before the labelled stage end. Directly measures operational usefulness.
Stage accuracy @ k	Top-1 and top-2 accuracy of predicted stage at $t+k$ vs actual, computed only over episodes where an attack occurred.
State forecast nRMSE	Per-feature error vs horizon. This is the metric that proves a world model was built — no classifier can even report it.
Calibration	Brier score and reliability diagram per horizon. A forecast saying 78% should be right about 78% of the time. Frequently overlooked and very persuasive to

7.4 Ablations — the falsification suite

Ablation	Procedure	Expected result
Persistence	Replace transition model with copy-forward	Substantial collapse in lead time and AUC. If it does not collapse, no dynamics were learned and we must say so.
Time-shuffle	Shuffle window order within each input sequence	Performance collapses. If it does not, the model is using per-window features only and the temporal claim is false.
Horizon curve	Plot AUC and nRMSE against k	Smooth degradation. Flat performance across all k is a red flag for leakage, not a good result.
Surprise signal	Compare state forecast error in benign vs pre-attack windows	Error rises before onset — attack onset is a dynamics regime change. A useful secondary detector and a strong plot.

7.5 Illustrative output format

```
Host 192.168.10.50 | t = 14:32:00 | window = 30s
Observed stage: Reconnaissance (confidence 0.91)

k   time      P(compromise)  90% band    Top predicted stage
+1  14:32:30    0.61          0.55-0.67   Reconnaissance  0.74
+2  14:33:00    0.69          0.60-0.77   Reconnaissance  0.58
+3  14:33:30    0.78          0.66-0.88   Initial Access  0.52
+4  14:34:00    0.83          0.68-0.94   Initial Access  0.61
+5  14:34:30    0.85          0.66-0.97   Lateral Movement 0.44

Threshold 0.75 crossed at k=3 -> FORECAST LEAD TIME 90 s
Simulated trajectories reaching Lateral Movement: 312 / 1000
```

8. Technology Stack

Layer	Choice	Rationale
Training	PyTorch 2.x	The rollout loop needs scheduled sampling, Gaussian NLL over predicted mean/log-variance, and a multi-step unrolled loss — all eager-mode custom training-step work. Natural in PyTorch, fighting the framework in Keras. Captum gives saliency directly.
Explainability	SHAP + Captum	KernelSHAP on the heads; input-gradient saliency for temporal attribution.
Backend	FastAPI + Uvicorn	Pydantic schema shared across services; native WebSocket for replay streaming; OpenAPI generates the frontend types.
Message bus	Redis Streams	Consumer groups, replay, and backpressure in ~30 lines and one container. Kafka would cost half a day of broker configuration for a

prototype that processes one PCAP — real cost, zero rubric credit.

Storage	PostgreSQL	Forecasts, episodes, computed metrics, lead-time distributions.
Frontend	Next.js + Recharts	The prediction-overlay moment needs charting control Streamlit fights you on. Conditional: only if a dedicated frontend person exists — otherwise Streamlit, which the statement explicitly permits, and spend the time on ablations instead.
Packaging	Docker Compose	Single <code>docker compose up</code> . Fully offline, no cloud dependency — satisfies the stated air-gap requirement.

9. Scalability & Deployment

This is where credit is earned on the architecture slide without paying the build cost. State the prototype topology and the production swaps explicitly, and support the claim with a measured number rather than an assertion.

Concern	Prototype	Production path
Message bus	Redis Streams, consumer groups	Kafka partitioned by host hash. Consumer-group semantics are equivalent, so this is a broker swap, not a rewrite.
Inference	1 replica, stateless	N replicas in one consumer group. Stateless design means linear scale-out with no coordination.
State history	PostgreSQL	TimescaleDB hypertables partitioned on window_ts.
Telemetry source	PCAP / CSV replay	SPAN port or NetFlow/IPFIX collector at the aggregation layer.
Throughput claim	Measure and report windows/sec/host from our own pipeline. An honest measured figure with a stated scale-out path is credible. A claimed Kafka cluster we never stress-tested is not.	

9.1 CII applicability

For Critical Information Infrastructure the relevant properties are: passive tap deployment (no inline failure mode), full air-gap operation with no external API calls, deterministic offline model loading from a pinned registry, and per-host forecasts that map to containment actions a SOC can actually take — isolate host, rate-limit egress, force re-auth — within the forecast lead time.

10. Build Plan

Phase	Deliverable	Exit criterion
Day 1-2 highest risk	Ingest + feature pipeline	A materialised [host, window, ~45 features] tensor for three CIC-IDS2017 days, with packet-level columns confirmed populated from tshark and normalisation stats fitted on the training period only. Most of the build time goes here, not into the model.
Day 3	World model + rollout	Training converges; K=6 recursive rollout produces decoded states; state nRMSE measurable at every horizon; frozen heads applied to predicted states.
Day 4	Evaluation harness	All four baselines run; persistence and time-shuffle ablations produce the expected collapse; median lead time computed; Infiltration holdout evaluated; calibration plotted.

Day 5	Services + UI + video	Docker Compose brings up the full offline stack; dashboard renders all six panels including flagged flows and the reality overlay; 2-minute demo recorded.
--------------	-----------------------	--

SEQUENCING WARNING

Packet-level extraction is the single highest-risk item and gates everything downstream. Start there on hour one. A perfect model on incomplete features fails the stated data requirement; a merely good model on complete features does not.

11. Risks

Risk	Severity	Mitigation
PCAP extraction too slow / too large	High	tshark field extraction rather than PyShark; bounded time ranges around labelled attacks plus a benign baseline; CIC-IDS2017 rather than 2018.
Rollout diverges past $k \approx 3$	Medium	Multi-step unrolled loss, scheduled sampling, ensembling. Report the horizon where calibration fails rather than hiding it — "reliable to $t+5$, degrades sharply after" is more credible than a claimed 20-step forecast.
Ablation shows no dynamics learned	High	This is a real possible outcome. If persistence matches the model, we report it honestly and diagnose — likely window size or context length. A negative result reported clearly beats a positive result that does not survive questioning.
Feature-name drift across services	Medium	Single Pydantic schema, <code>FEATURE_ORDER</code> constant, generated TS types, schema version on every message.
Scope creep into graph modelling	Medium	Graph is a feature extractor. Temporal GNN is explicitly out of scope for v1.
Frontend consumes model time	Low	Streamlit fallback is pre-agreed and rubric-compliant.

12. Scientific Honesty & Claims Discipline

The problem statement describes world models as learning "an internal causal simulation". We adopt that framing in the pitch and are precise in the technical document. Both can be true; the distinction is where technical judges probe hardest.

Concept	Do we have it?	Basis
Correlation	Yes	Features co-occur with attack labels. Says nothing about direction or mechanism.
Temporal prediction	Yes	<code>S_t</code> precedes and is informative about <code>S_{t+1}</code> . Direction in time is a constraint, not a causal warrant.
Learned dynamics	Yes	The model captures regularities in how the system evolves, sufficient to simulate forward. A legitimate and non-trivial claim.
Causal inference	No	Requires intervention, randomisation, or identification under an explicit causal graph. Our data is observational and confounded by collection protocol — in CIC-IDS2017 attacks were scripted at scheduled times, so time-of-day correlates with attack presence as an artefact of the experiment.

Where we sit on the world-model ladder

A sequence classifier is not a world model. A single-step next-state predictor is minimal but real. **We target a latent-state transition model with calibrated multi-step rollout** — architecturally a world model, operationally a forecaster. We do **not** claim a full generative, action-conditioned world model: that requires an action space and interventional data the public datasets cannot provide. Claiming it invites a question we cannot answer, and being caught overclaiming costs far more than scoping honestly.

LANGUAGE DISCIPLINE

Use: learned dynamics, forecast, simulated trajectory, projected state, model-internal counterfactual.
Avoid: causal simulator, "the model understands why", "predicts attacker intent".
Counterfactual rollouts answer "what would this model predict if the input were different" — a question about the model, not about the network. Labelling that distinction unprompted signals rigour that is rare and memorable.

13. Deliverables Mapping

Required deliverable	Where it is satisfied
Feature extraction pipeline (flow + packet)	§2 Ingest + Feature services, Figure 1. Both levels mandatory per §4.1.
Trained world model demonstrably learning dynamics	§3, Figure 2. "Demonstrably" is answered by the §7.4 ablation suite and state nRMSE.
Training scripts, weights, reproducible config	Model registry: pinned seeds, <code>config.yaml</code> , checkpoints, serialised scaler. One-command reproduction in README.
K-step infiltration prediction engine	§3.2, §5.1 <code>Forecast</code> schema.
ATT&CK stage mapping	Frozen stage head, §3.1. Presented as a curated tactic mapping over dataset labels — not technique-level ground truth.
Explainability (SHAP / attention)	§6. Hard gate — "black-box outputs are not acceptable". Must be visible in the demo video, not buried in a notebook.
Offline demo interface (PCAP/CSV in)	§5.3. Displays probability timeline, flagged flows, stage annotations as specified.
Benchmark vs logistic regression	§7.2. LR on identical features is the headline row; three further baselines run internally.
Architecture doc (2 pp) · Deck (5 slides) · Video (2 min)	Compressed from this PRD — allocation in §14.

14. Demo & Presentation

14.1 The two-minute demo

Time	Beat
0:00-0:20	Split screen. Left: CIC-IDS2017 Wednesday replaying, a host's feature vector updating each window. Right: risk timeline flat and green near 0.08. A conventional IDS strip along the bottom, silent. State forecast error steady and low.
0:20-0:40	Reconnaissance begins. Port entropy and fan-out climb. The state forecast error line spikes first — the dynamics model is surprised by a regime change. Risk begins to lift moments later. The IDS strip is

still silent.

0:40-1:05	The forecast panel unfolds: six ghost windows extend right of "now", bands widening with distance, t+3 crossing 0.75 and turning amber, stage at t+4 flipping to Initial Access. Explanation populates with five signed signals. Caption: forecast lead time 90 seconds .
1:05-1:25	Counterfactual. Click "hold new_peer_count constant", re-roll. Projected curve drops 0.85 → 0.31, amber marker clears. "The model's internal simulation says fan-out growth drives this trajectory — that is where containment goes."
1:25-1:45 the proof	Fast-forward to real t+3. Actual traffic arrives. Overlay observed state on the predicted ghost state — they match. The IDS strip fires now, ninety seconds late . Freeze frame with both timestamps visible.
1:45-2:00	Results: LR-on-history vs model, persistence ablation collapse, horizon-vs-AUC curve, Infiltration holdout, median lead time with distribution.

Everything before 1:25 is a claim. The overlay of prediction on subsequently observed reality is the proof. Protect that shot in the edit.

14.2 Five-slide allocation

1. Problem, and the detection-vs-forecasting distinction (the §1.3 table, trimmed)
2. Architecture (Figure 1, simplified)
3. World model and rollout, with the worked lead-time example (Figure 2 + §7.5)
4. Benchmarks vs logistic regression, plus the ablation collapse
5. Generalisation, CII deployment path, and honest limits

14.3 Two-page architecture document

Pipeline diagram · state definition · transition model and rollout · evaluation table including the persistence ablation. Nothing else fits, and nothing else is load-bearing.

15. One-Paragraph Summary

We are not building an intrusion detector. We chop network traffic into 30-second windows and, for each host, compute a vector describing what that host was doing in that window — flow statistics, packet statistics, and graph-derived numbers such as how many new peers it contacted. That vector is the network state. We train a model whose job is to predict the *next* state vector from the history of previous ones, so it learns how network behaviour evolves rather than what a label looks like. Once it can do that, we let it run forward on its own imagination for six windows, producing states nobody observed. Then we apply a separately trained, frozen risk classifier to those imagined states. Because that classifier only ever saw real states during training, any ability to warn early comes entirely from the dynamics model — which is exactly the thing we can prove by replacing the dynamics with copy-the-last-state and watching performance collapse. The output is a risk curve over the next three minutes with uncertainty bands, a predicted attack stage, the top signals driving it, and a lead time in seconds.